Mirror or Window? A Student Study on LLM-Generated Reflection Questions for Self vs. Peer Code

This notebook conducts an in-depth analysis of data from the study "**Mirror or Window? A Student Study on LLM-Generated Reflection Questions for Self vs. Peer Code**". The goal was to understand the impact of Questions on Learner's Code (QLCs) on student performance and perceptions. The analysis covers various aspects including student participation, exercise completion, scores, time spent, and feedback ratings, culminating in an examination of exam scores and pass percentages.

Installation + Imports

This section handles the installation of necessary libraries and imports all required modules for data manipulation, visualization, and statistical analysis. Warnings are filtered to keep the output clean.

```
!pip -q install pandas numpy seaborn matplotlib scipy statsmodels pingouin

import json
import math
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from scipy.stats import ttest_ind, mannwhitneyu, chi2_contingency
import statsmodels.formula.api as smf

from google.colab import userdata

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10,6)

user_id_column_name = userdata.get("idCol") # The name of the column of user ids is tied to an affiliated instiution
```

Here, custom utility functions are defined to assist with loading JSON data, converting MongoDB-specific values (like `$oid` and `$date`), parsing dates, and safely calculating the length of list-like objects. These functions streamline the data ingestion and initial cleaning process.

```
def load_json_lines(filename):
    """
    Reads a JSON file and turns it to text
    Supports:
    - JSON array
    - newline-delimited json
    """
    with open(filename, "r", encoding="utf-8") as f:
        text = f.read().strip()

    try:
        data = json.loads(text)
        if isinstance(data, list):
            return data
        else:
            return [data]
    except:
        rows = []
        for line in text.splitlines():
            line = line.strip()
            if line:
                rows.append(json.loads(line))
        return rows


def mongo_val(x):
    """
    Convert common Mongo export wrappers:
    {"$oid": "..."}
    {"$date": "..."}
    """
    if isinstance(x, dict):
        if "$oid" in x:
            return x["$oid"]
```

```
        if "$date" in x:
            return x["$date"]
    return x


def parse_date(x):
    x = mongo_val(x)
    return pd.to_datetime(x, errors="coerce")


def safe_len(x):
    if isinstance(x, list):
        return len(x)
    return 0
```

The raw data for users, exercises, and assignments are loaded from JSON files. These files contain detailed records of student interactions, exercise metadata, and assignment details, forming the foundation of our analysis.

```
import pandas as pd
import numpy as np

# Raw data loading
raw_users_data = load_json_lines("qlcs.users.json")
raw_exercises_data = load_json_lines("qlcs.exercises.json")
raw_assignments_data = load_json_lines("qlcs.assignments.json")

# 1. Create a DataFrame for all users, including non-participants
all_users_df = pd.DataFrame(raw_users_data)
for c in all_users_df.columns:
    all_users_df[c] = all_users_df[c].apply(mongo_val)
all_users_df["studyParticipation"] = all_users_df["studyParticipation"].fillna(False)
all_users_df = all_users_df.rename(columns={user_id_column_name: "userId", "studyGroup": "assigned_group"})
# Drop PII columns that are not used in subsequent analysis
all_users_df = all_users_df.drop(columns=['firstName', 'lastName', 'email'], errors='ignore')

# Filter out non-student roles
all_users_df = all_users_df[all_users_df["role"] == "student"].copy()
# 2. Identify students who did NOT have studyParticipation = True
non_participating_students = all_users_df[all_users_df["studyParticipation"] == False].copy()
print(f"Total students who did not have studyParticipation = True: {len(non_participating_students)}")
# 3. Create a DataFrame for all exercises, and link them to all users.
all_ex_rows = []
for row in raw_exercises_data:
    r = {}
    for k, v in row.items():
        if k != "questions":
            r[k] = mongo_val(v)
    r["questions"] = row.get("questions", [])
    all_ex_rows.append(r)
all_exercises_df = pd.DataFrame(all_ex_rows)
all_exercises_df["createdTimestamp"] = all_exercises_df["createdTimestamp"].apply(parse_date)
all_exercises_df["completedTimestamp"] = all_exercises_df["completedTimestamp"].apply(parse_date)
# Now merge all_exercises_df with all_users_df to get studyParticipation status for each exercise entry
exercises_with_all_user_info = all_exercises_df.merge(
    all_users_df[['userId', 'studyParticipation']],
    on="userId",
    how="left"
)
# 4. Count how many in this group of non-participating students completed at least 1 exercise
completed_exercises_by_non_participants = exercises_with_all_user_info[
    (exercises_with_all_user_info["studyParticipation"] == False) &
    (exercises_with_all_user_info["status"] == "Complete")
].copy()
students_completed_at_least_one_exercise = completed_exercises_by_non_participants["userId"].nunique()
user_ids_to_remove = completed_exercises_by_non_participants["userId"].unique()
print(f"Number of non-participating students who completed at least 1 exercise: {students_completed_at_least_one_exercise}")

study_participants_df = pd.DataFrame(raw_users_data)
for c in study_participants_df.columns:
    study_participants_df[c] = study_participants_df[c].apply(mongo_val)
study_participants_df["studyParticipation"] = study_participants_df["studyParticipation"].fillna(False)
study_participants_df = study_participants_df[
    (study_participants_df["studyParticipation"] == True) &
    (study_participants_df["role"] == "student")
].copy()
study_participants_df = study_participants_df.rename(columns={
    user_id_column_name:"userId",
    "studyGroup":"assigned_group"
})
study_participants_df = study_participants_df[[
    "userId","assigned_group"
]]
print("Opted into study users:", len(study_participants_df))
```

```
assignments_df = pd.DataFrame(raw_assignments_data)
for c in assignments_df.columns:
    assignments_df[c] = assignments_df[c].apply(mongo_val)
assignments_df["dueDate"] = assignments_df["dueDate"].apply(parse_date)
assignments_df = assignments_df.rename(columns={
    "uuid":"assignmentId"
})
assignments_df = assignments_df[[
    "assignmentId","chapterId","title","identifier","dueDate"
]]
assignments_df = assignments_df.sort_values("dueDate").reset_index(drop=True)
assignments_df["assignment_order"] = np.arange(1, len(assignments_df)+1)

ex_rows = []
for row in raw_exercises_data:
    r = {}
    for k,v in row.items():
        if k != "questions":
            r[k] = mongo_val(v)
    r["questions"] = row.get("questions", [])
    ex_rows.append(r)
exercises_df = pd.DataFrame(ex_rows)
exercises_df["createdTimestamp"] = exercises_df["createdTimestamp"].apply(parse_date)
exercises_df["completedTimestamp"] = exercises_df["completedTimestamp"].apply(parse_date)
```

```
Total students who did not have studyParticipation = True: 10
Number of non-participating students who completed at least 1 exercise: 3
Opted into study users: 43
```

Data Preparation and Filtering

This crucial section focuses on processing the raw JSON data into structured Pandas DataFrames. It involves initial cleaning, filtering out irrelevant entries (e.g., non-student roles, non-participating students), and merging various datasets to create a comprehensive view of student activities.

Merging JSONs

This block consolidates the processing of raw data from `qlcs.users.json`, `qlcs.exercises.json`, and `qlcs.assignments.json` to create several key DataFrames:

1. **`all_users_df`**: Contains all student roles, with `mongo_val` applied and `studyParticipation` filled. It also identifies `user_ids_to_remove` for non-participating students who completed exercises.
2. **`all_exercises_df`**: Contains all exercise data, with timestamps parsed and merged with `all_users_df` to include `studyParticipation` status.
3. **`study_participants_df`**: Filters `raw_users_data` to include only students who opted into the study (`studyParticipation == True`) and have a 'student' role. Columns are renamed for consistency.
4. **`assignments_df`**: Processes `raw_assignments_data`, converting MongoDB values, parsing due dates, renaming `uuid` to `assignmentId`, and adding `assignment_order`.
5. **`exercises_df`**: Processes `raw_exercises_data`, extracting relevant fields and parsing timestamps. This serves as the base for further exercise-related analysis.

This consolidation streamlines the initial data preparation, making the notebook more efficient and easier to understand.

The following cells meticulously filter the exercise data to ensure only relevant and unique submissions from study-participating students are included in the analysis. This involves several steps:

1. **Filtering by Study Participation**: Only exercises from students who opted into the study (Group A or B) and whose `authorId` also belongs to an eligible student are kept.
2. **Deduplication**: In cases where exercises were regenerated, only the most recent instance (`createdTimestamp`) for each `userId` and `assignmentId` combination is retained.
3. **Assignment Filtering**: Exercises related to specific assignments (PA06-PA09) are selected, excluding any assignments that were disrupted or incomplete (e.g., PA10).

This cell merges the `ex` DataFrame with the `users` DataFrame based on `userId`. It also ensures that the `authorId` of the exercises also belongs to an eligible study participant. This step is critical for aligning exercises with the correct study groups and filtering out exercises from non-participants.

```
# Filter where studyParticipation = True
exercises_df = exercises_df.merge(study_participants_df, on="userId", how="inner")

# Get a set of eligible user IDs (those with studyParticipation = true)
eligible_user_ids = study_participants_df['userId'].unique()
```

```
# Filter 'exercises_df' to ensure authorId also belongs to an eligible user
exercises_df = exercises_df[exercises_df['authorId'].isin(eligible_user_ids)].copy()

print("Exercises after user and author filter:", len(exercises_df))
```

```
Exercises after user and author filter: 565
```

Exercises are often regenerated by students. This code block sorts exercises by `createdTimestamp` and then uses `groupby().tail(1)` to keep only the most recent version of each exercise submission for a given `userId` and `assignmentId`. This ensures that analysis is based on the final attempt.

```
exercises_df = exercises_df.sort_values("createdTimestamp")

exercises_df["regen_count"] = exercises_df.groupby(
    ["userId","assignmentId"]
)["assignmentId"].transform("count")

latest_exercises_df = exercises_df.groupby(
    ["userId","assignmentId"],
    as_index=False
).tail(1).copy()

print("After dedupe:", len(latest_exercises_df))
```

```
After dedupe: 513
```

This cell performs the final filtering of exercises. It merges the `ex_latest` DataFrame with assignment details (`assign`) to incorporate `dueDate` and `chapterId`. It then identifies and removes exercises from a 'disrupted chapter' (PA10), focusing the analysis on assignments PA06-PA09 as specified.

```
processed_exercises_df = latest_exercises_df.merge(assignments_df, on="assignmentId", how="left")
latest_due = processed_exercises_df["dueDate"].max()

final_chapter_ids = processed_exercises_df.loc[
    processed_exercises_df["dueDate"] == latest_due, "chapterId"
].unique()

processed_exercises_df["disrupted_chapter"] = processed_exercises_df["chapterId"].isin(final_chapter_ids).astype(int)
processed_exercises_df = processed_exercises_df[processed_exercises_df["disrupted_chapter"] == 0].copy()
print("# of total exercises, PA06-PA09:", len(processed_exercises_df))
```

```
# of total exercises, PA06-PA09: 427
```

Regeneration attempts

This cell calculates the total number of regeneration attempts by subtracting the count of unique, latest exercises (after deduplication and chapter filtering) from the count of all exercises before deduplication and chapter filtering. This gives an aggregate measure of how often students regenerated their exercises.

```
# Create a temporary DataFrame from the original exercises_df,
# merged with assignments_df to get chapterId, then filter out disrupted chapters.
exercises_pre_dedupe_filtered_df = exercises_df.merge(
    assignments_df[['assignmentId', 'chapterId']],
    on='assignmentId',
    how='left'
)
exercises_pre_dedupe_filtered_df = exercises_pre_dedupe_filtered_df[
    ~exercises_pre_dedupe_filtered_df['chapterId'].isin(final_chapter_ids)
].copy()

# processed_exercises_df already contains the latest exercises merged with assignment info and filtered for disrupted chapters

# Recalculate total_regenerations with the filtered DataFrames
total_regenerations = len(exercises_pre_dedupe_filtered_df) - len(processed_exercises_df)
print(f"Total instances where exercises were regenerated (excluding disrupted chapter): {total_regenerations}")
```

```
Total instances where exercises were regenerated (excluding disrupted chapter): 17
```

This section creates a lookup table (`order_labels`) to map assignment order to their identifiers. This is useful for consistent plotting and ordering of assignments in visualizations.

```
order_labels = processed_exercises_df[['assignment_order', 'identifier']].drop_duplicates().sort_values('assignment_order').reset_index(drop=Tr
```

Students opted in, no exercises

These are students who signed the consent form, but didn't complete a single exercise.

This code block calculates various counts related to completed and assigned exercises, differentiating between 'Own' (authored by self) and 'Other' (authored by another student) conditions. It then merges these counts into a `summary_table` for each user and identifies students who completed zero exercises, indicating a lack of engagement despite opting in.

```
processed_exercises_df["actual_condition"] = np.where(
    processed_exercises_df["authorId"] == processed_exercises_df["userId"],
    "Own",
    "Other"
)
completed_exercises_df = processed_exercises_df[processed_exercises_df["status"] == "Complete"].copy()

# 1. Total exercises completed by each user
total_completed = completed_exercises_df.groupby(['userId', 'assigned_group']).size().reset_index(name='Total Completed Exercises')

# 2. Exercises completed where authorId == userId (authored by themselves)
own_authored = completed_exercises_df[completed_exercises_df['actual_condition'] == 'Own']
own_authored_count = own_authored.groupby(['userId', 'assigned_group']).size().reset_index(name='Completed Own Authored')

# 3. Exercises completed where authorId != userId (authored by others)
other_authored = completed_exercises_df[completed_exercises_df['actual_condition'] == 'Other']
other_authored_count = other_authored.groupby(['userId', 'assigned_group']).size().reset_index(name='Completed Other Authored')

# 4. Count of assigned exercises where authorId == userId (authored by themselves)
assigned_own_authored = processed_exercises_df[processed_exercises_df['actual_condition'] == 'Own']
assigned_own_authored_count = assigned_own_authored.groupby(['userId', 'assigned_group']).size().reset_index(name='Assigned Own Authored')

# 5. Count of assigned exercises where authorId != userId (authored by others)
assigned_other_authored = processed_exercises_df[processed_exercises_df['actual_condition'] == 'Other']
assigned_other_authored_count = assigned_other_authored.groupby(['userId', 'assigned_group']).size().reset_index(name='Assigned Other Authored'

# Initialize participation_summary_table_df with all study participants to ensure all participants are included even if they didn't complete ex
participation_summary_table_df = study_participants_df.copy()

# Merge each count DataFrame using a left merge to retain all users
participation_summary_table_df = participation_summary_table_df.merge(total_completed, on=['userId', 'assigned_group'], how='left')
participation_summary_table_df = participation_summary_table_df.merge(own_authored_count, on=['userId', 'assigned_group'], how='left')
participation_summary_table_df = participation_summary_table_df.merge(other_authored_count, on=['userId', 'assigned_group'], how='left')
participation_summary_table_df = participation_summary_table_df.merge(assigned_own_authored_count, on=['userId', 'assigned_group'], how='left')
participation_summary_table_df = participation_summary_table_df.merge(assigned_other_authored_count, on=['userId', 'assigned_group'], how='left

# Fill NaN values with 0 for users who might not have completed/assigned any exercises in a specific category
participation_summary_table_df[['Total Completed Exercises', 'Completed Own Authored', 'Completed Other Authored', 'Assigned Own Authored', 'As
    participation_summary_table_df[['Total Completed Exercises', 'Completed Own Authored', 'Completed Other Authored', 'Assigned Own Authored',

students_zero_exercises = participation_summary_table_df[participation_summary_table_df['Total Completed Exercises'] == 0]
print(f"Number of students who completed zero exercises: {len(students_zero_exercises)}")
```

```
Number of students who completed zero exercises: 8
```

This section further explores the interaction counts, specifically looking at how many times students in Group A received and completed an exercise from another student (Self), and similarly for Group B receiving and completing their own exercise (Peer). It also displays the comprehensive `summary_table` detailing engagement per student.

Feature Engineering and Completion Analysis

This section enriches the `df` DataFrame with new features derived from existing data, such as score percentage, completion status, average time per question, and a `compliant` flag indicating whether the student adhered to their assigned study condition. It also prepares `df_complete` for analysis of completed exercises.

```
# 1. Total number of assigned exercises across the listed students
total_assigned_exercises = participation_summary_table_df['Assigned Own Authored'].sum() + participation_summary_table_df['Assigned Other Autho

# 2. Sum of the 'Assigned Other Authored' column, filtering by instances where assigned_group is A
assigned_other_authored_group_A = participation_summary_table_df[participation_summary_table_df['assigned_group'] == 'A']['Assigned Other Autho
completed_other_authored_group_A = participation_summary_table_df[participation_summary_table_df['assigned_group'] == 'A']['Completed Other Aut
print(f"Group A instances where a student received another student's exercise and completed it: {completed_other_authored_group_A}")
print(f"Group A instances where a student received another student's exercise: {assigned_other_authored_group_A}")

# 3. Sum of the 'Assigned Own Authored' column, filtering by instances where assigned_group is B
assigned_own_authored_group_B = participation_summary_table_df[participation_summary_table_df['assigned_group'] == 'B']['Assigned Own Authored'
completed_own_authored_group_B = participation_summary_table_df[participation_summary_table_df['assigned_group'] == 'B']['Completed Own Authore
print(f"Group B instances where a student received their own exercise and completed it: {completed_own_authored_group_B}")
print(f"Group B instances where a student received their own exercise: {assigned_own_authored_group_B}")
```

```
# Display the table, sorted by assigned_group for readability and without userId
display(participation_summary_table_df.drop(columns=['userId']).sort_values(by='assigned_group'))
```

```
Group A instances where a student received another student's exercise and completed it: 8
Group A instances where a student received another student's exercise: 33
Group B instances where a student received their own exercise and completed it: 0
Group B instances where a student received their own exercise: 0
```

	assigned_group	Total Completed Exercises	Completed Own Authored	Completed Other Authored	Assigned Own Authored	Assigned Other Authored
14	A	0	0	0	0	10
31	A	10	10	0	10	0
24	A	0	0	0	5	5
22	A	6	4	2	6	4
34	A	10	10	0	10	0
19	A	10	10	0	10	0
18	A	0	0	0	6	4
17	A	3	3	0	10	0
16	A	10	10	0	10	0
15	A	10	10	0	10	0
27	A	1	1	0	8	2
13	A	10	10	0	10	0
29	A	10	8	2	8	2
10	A	8	8	0	9	1
9	A	3	2	1	9	1
6	A	0	0	0	10	0
38	A	10	10	0	10	0
3	A	10	10	0	10	0
2	A	10	9	1	9	1
40	A	0	0	0	9	1
12	A	10	8	2	8	2
35	B	0	0	0	0	10
36	B	6	0	6	0	10
37	B	10	0	10	0	10
39	B	10	0	10	0	10
33	B	5	0	5	0	10
32	B	0	0	0	0	10
30	B	10	0	10	0	10
0	B	10	0	10	0	10
21	B	8	0	8	0	10
26	B	10	0	10	0	10
25	B	1	0	1	0	10
23	B	10	0	10	0	10
41	B	0	0	0	0	7
20	B	8	0	8	0	10
11	B	9	0	9	0	10
8	B	10	0	10	0	10
7	B	1	0	1	0	10
5	B	3	0	3	0	10
4	B	4	0	4	0	10
1	B	10	0	10	0	10
28	B	10	0	10	0	10
42	B	10	0	10	0	10

Assemble Completed Exercises

This cell calculates several new features for each exercise entry in the `df` DataFrame:

- `num_questions`: The number of questions in the exercise.
- `score_pct`: The percentage of correct answers.
- `completion_binary`: A binary indicator (1 if 'Complete', 0 otherwise).
- `avg_time_per_question`: Average time spent per question.
- `actual_condition`: Whether the exercise was 'Own' (authored by self) or 'Other' (authored by another student).
- `compliant`: A binary flag indicating if the student's actual condition matches their assigned group (Group A with 'Own' or Group B with 'Other').
- `days_before_due`: Number of days between exercise creation and its due date.

Finally, it creates `df_complete` containing only completed exercises and prints counts.

```
processed_exercises_df["num_questions"] = processed_exercises_df["questions"].apply(safe_len)

processed_exercises_df["score_pct"] = np.where(
    processed_exercises_df["num_questions"] > 0,
    processed_exercises_df["totalCorrect"] / processed_exercises_df["num_questions"],
    np.nan
)

processed_exercises_df["completion_binary"] = (processed_exercises_df["status"] == "Complete").astype(int)

processed_exercises_df["avg_time_per_question"] = np.where(
    processed_exercises_df["num_questions"] > 0,
    processed_exercises_df["totalTimeSpent"] / processed_exercises_df["num_questions"],
    np.nan
)

processed_exercises_df["actual_condition"] = np.where(
    processed_exercises_df["authorId"] == processed_exercises_df["userId"],
    "Own",
    "Other"
)

processed_exercises_df["compliant"] = np.where(
    ((processed_exercises_df["assigned_group"]=="A") & (processed_exercises_df["actual_condition"]=="Own")) |
    ((processed_exercises_df["assigned_group"]=="B") & (processed_exercises_df["actual_condition"]=="Other")),
    1,0
)

processed_exercises_df["days_before_due"] = (
    processed_exercises_df["dueDate"] - processed_exercises_df["createdTimestamp"]
).dt.total_seconds() / (60*60*24)

completed_exercises_df = processed_exercises_df[processed_exercises_df["status"] == "Complete"].copy()

print("# of assigned exercises:", processed_exercises_df.shape[0])
print("# of completed exercises:", completed_exercises_df.shape[0])

group_palette = {
    "A": "#1f77b4",  # blue
    "B": "#ff7f0e"   # orange
}
```

```
# of assigned exercises: 427
# of completed exercises: 276
```

Score Distribution

This cell prepares the data for visualizing score distributions. It rounds `score_pct` to create discrete `score_label` categories (0%, 20%, ..., 100%). It then calculates both raw counts and percentages of exercises falling into each score category, grouped by `assigned_group`.

```
score_distribution_df = completed_exercises_df.copy()

# If score_pct already equals 0.0,0.2,...1.0 this is fine
score_labels = {
    0.0: "0%",
    0.2: "20%",
    0.4: "40%",
    0.6: "60%",
    0.8: "80%",
    1.0: "100%"
```

```
}

score_distribution_df["score_label"] = score_distribution_df["score_pct"].round(1).map(score_labels)

# Counts
counts = pd.crosstab(
    score_distribution_df["score_label"],
    score_distribution_df["assigned_group"]
)

# Percentages within each group
pct = pd.crosstab(
    score_distribution_df["score_label"],
    score_distribution_df["assigned_group"],
    normalize="columns"
) * 100

# Order rows
row_order = ["0%", "20%", "40%", "60%", "80%", "100%"]
pct = pct.reindex(row_order)
counts = counts.reindex(row_order)

# Combine nicely
summary = pct.round(1).astype(str) + "% (n=" + counts.fillna(0).astype(int).astype(str) + ")"

display(summary)
```

assigned_group	A	B
score_label		
0%	1.5% (n=2)	6.9% (n=10)
20%	15.3% (n=20)	9.0% (n=13)
40%	12.2% (n=16)	11.0% (n=16)
60%	23.7% (n=31)	23.4% (n=34)
80%	22.9% (n=30)	32.4% (n=47)
100%	24.4% (n=32)	17.2% (n=25)

This cell further processes `completed_exercises_df` to create a `completed_exercises_grouped_by_assignment_df` DataFrame. It extracts a simplified `assignment_group_id` (e.g., 'PA06' from 'PA06-A') for consistent plotting.

```
completed_exercises_grouped_by_assignment_df = completed_exercises_df.copy()

# Extract the simplified chapter identifier (e.g., 'PA06')
# using the 'identifier' column which came from the 'assign' DataFrame.
completed_exercises_grouped_by_assignment_df["assignment_group_id"] = completed_exercises_grouped_by_assignment_df["identifier"].apply(
    lambda x: x.split("-")[0] if isinstance(x, str) and "-" in x else x
)

# Recalculate order_labels to use the new chapter IDs
# Group by assignment_order and assignment_group_id, then get the first identifier
order_labels_grouped = (
    completed_exercises_grouped_by_assignment_df[["assignment_order", "assignment_group_id"]]
    .drop_duplicates()
    .sort_values("assignment_order")
)
```

This cell calculates and prints the overall average and standard deviation of `score_pct` for both Group A and Group B, providing a summary of the average performance across all completed exercises. It also computes average scores for each assignment and group.

```
# Calculate and print overall average score for each group
overall_avg_group_A = completed_exercises_df[completed_exercises_df['assigned_group'] == 'A']['score_pct'].mean()
overall_std_group_A = completed_exercises_df[completed_exercises_df['assigned_group'] == 'A']['score_pct'].std()
overall_avg_group_B = completed_exercises_df[completed_exercises_df['assigned_group'] == 'B']['score_pct'].mean()
overall_std_group_B = completed_exercises_df[completed_exercises_df['assigned_group'] == 'B']['score_pct'].std()
print(f"Overall Average Score Percentage for Group A: {overall_avg_group_A:.2f} (Std Dev: {overall_std_group_A:.2f})")
print(f"Overall Average Score Percentage for Group B: {overall_avg_group_B:.2f} (Std Dev: {overall_std_group_B:.2f})")

# Group by assigned_group and chapter identifier, then calculate the mean of 'score_pct'
# For a more granular view, we can keep the full 'identifier' which implies chapter and sub-assignment
average_scores_by_chapter_group = completed_exercises_df.groupby(['assigned_group', 'identifier'])['score_pct'].mean().reset_index()

# Rename columns for clarity
average_scores_by_chapter_group.rename(columns={'identifier': 'Chapter_Assignment_Identifier', 'score_pct': 'Average_Score_Percentage'}, inplac

# Sort for better readability
average_scores_by_chapter_group_sorted = average_scores_by_chapter_group.sort_values(by=['assigned_group', 'Chapter_Assignment_Identifier'])
```

```
# display(average_scores_by_chapter_group_sorted)
```

```
Overall Average Score Percentage for Group A: 0.65 (Std Dev: 0.29)
Overall Average Score Percentage for Group B: 0.63 (Std Dev: 0.29)
```

Time Spent

This cell performs a similar aggregate analysis of `totalTimeSpent` as the previous one, but *without* filtering for outliers. This allows for a comparison to see if outlier removal significantly changes the group differences in total time spent.

```
# Use completed_exercises_df directly without filtering for outliers
time_analysis_df = completed_exercises_df.copy()

# Extract the simplified assignment identifier (e.g., 'PA06' from 'PA06-A')
time_analysis_df["assignment_group_id"] = time_analysis_df["identifier"].apply(
    lambda x: x.split("-")[0] if isinstance(x, str) and "-" in x else x
)

# Recalculate order_labels to use the new grouped assignment IDs for the full data
# This part is kept as it might be used elsewhere or for future granular analysis if needed,
# but it won't be used for the aggregate time spent analysis in this cell.
order_labels_grouped = (
    time_analysis_df[["assignment_order", "assignment_group_id"]]
    .drop_duplicates()
    .sort_values("assignment_order")
)

print("\n--- Aggregate Summary of Total Time Spent (All Data) ---")
# Aggregate summary by assigned_group only, removing assignment_group_id
time_spent_summary_aggregate = time_analysis_df.groupby('assigned_group')['totalTimeSpent'].agg(['mean', 'std', 'count']).round(2)
display(time_spent_summary_aggregate)

print("\n--- Independent T-test for Aggregate Total Time Spent between Self and Peer groups ---")
alpha = 0.05

# Prepare data for the aggregate t-test
time_group_a_aggregate = time_analysis_df[
    time_analysis_df['assigned_group'] == 'A'
]['totalTimeSpent']

time_group_b_aggregate = time_analysis_df[
    time_analysis_df['assigned_group'] == 'B'
]['totalTimeSpent']

# Perform a single aggregate t-test
if len(time_group_a_aggregate) > 1 and len(time_group_b_aggregate) > 1: # Need at least 2 data points for t-test
    stat, p = ttest_ind(time_group_a_aggregate, time_group_b_aggregate, equal_var=False) # Welch's t-test for unequal variances
    print(f"  Aggregate T-test: t={stat:.3f}, p={p:.3f}")
    if p < alpha:
        print(f"  There is a statistically significant difference (p < {alpha}).")
    else:
        print(f"  There is no statistically significant difference (p >= {alpha}).")
else:
    print(f"  Not enough data to perform an aggregate t-test.")
```

```
--- Aggregate Summary of Total Time Spent (All Data) ---
```

assigned_group	mean	std	count
A	417.61	413.65	131
B	415.31	346.20	145

```
--- Independent T-test for Aggregate Total Time Spent between Self and Peer groups ---
  Aggregate T-test: t=0.050, p=0.960
  There is no statistically significant difference (p >= 0.05).
```

Individual Questions

This code block iterates through each completed exercise in `completed_exercises_df` and then through each individual question within those exercises. It extracts question-specific details like `uuid`, `type`, `correctness`, `status`, `timeSpent`, `attempts`, and `num_hints`, and compiles them into a new DataFrame called `individual_questions_df`. This DataFrame forms the basis for question-level analysis.

```
q_rows_complete = []
```

```
for _, row in completed_exercises_df.iterrows():

    qs = row["questions"]

    if not isinstance(qs, list):
        continue

    for q in qs:

        qq = {}

        qq["userId"] = row["userId"]
        qq["assignmentId"] = row["assignmentId"]
        qq["assigned_group"] = row["assigned_group"]

        qq["actual_condition"] = row["actual_condition"]

        qq["question_uuid"] = mongo_val(q.get("uuid"))
        qq["question_type"] = q.get("type")
        qq["correct"] = q.get("correct")
        qq["status"] = q.get("status")
        qq["timeSpent"] = q.get("timeSpent", np.nan)

        qq["attempts"] = safe_len(q.get("userAnswers", []))
        qq["num_hints"] = safe_len(q.get("hints", []))

        q_rows_complete.append(qq)

individual_questions_df = pd.DataFrame(q_rows_complete)

print("Question rows from completed exercises:", len(individual_questions_df))
```

```
Question rows from completed exercises: 1380
```

The `individual_questions_df` DataFrame is aggregated here by `question_uuid` to calculate the mean `correctness`, `attempts`, and `timeSpent` for each unique question, creating `aggregated_questions_df`. The results are sorted by `correctness`, providing an overview of question difficulty and engagement.

```
aggregated_questions_df = individual_questions_df.groupby("question_uuid").agg({
    "correct":"mean",
    "attempts":"mean",
    "timeSpent":"mean"
}).sort_values("correct")

print("Aggregated question data from completed exercises (first 5 rows, omitting specific question IDs):")
display(aggregated_questions_df.reset_index(drop=True).head())
```

```
Aggregated question data from completed exercises (first 5 rows, omitting specific question IDs):
```

	correct	attempts	timeSpent
0	0.0	3.0	5.0
1	0.0	2.0	5.0
2	0.0	2.0	58.0
3	0.0	2.0	7.0
4	0.0	4.0	3.0

This extensive cell performs statistical analyses on `timeSpent` per question. It calculates mean and standard deviation of `timeSpent` per group. Crucially, it conducts two types of t-tests:

1. **Overall T-test**: Compares the aggregated `timeSpent` per question between Group A and Group B across all questions.
2. **Individual Question T-tests**: Iterates through each unique question and performs a separate t-test to see if there's a significant difference in `timeSpent` between Group A and Group B for *that specific question*.

```
print("--- Average Time Spent per Question by Group ---")
# Calculate the mean and standard deviation of timeSpent for each assigned_group
question_time_summary_df = individual_questions_df.groupby('assigned_group')['timeSpent'].agg(['mean', 'std'])

display(question_time_summary_df)

print("Average time spent per question by group calculated.")

print("\n--- Overall T-test for Time Spent per Question between Group A and Group B ---")
alpha = 0.05

# Prepare data for the overall t-test on 'timeSpent' across all questions
time_group_a_overall = individual_questions_df[
    individual_questions_df['assigned_group'] == 'A'
]['timeSpent'].dropna()
```

```
time_group_b_overall = individual_questions_df[
    individual_questions_df['assigned_group'] == 'B'
]['timeSpent'].dropna()

if len(time_group_a_overall) > 1 and len(time_group_b_overall) > 1:
    stat_overall, p_val_overall = ttest_ind(time_group_a_overall, time_group_b_overall, equal_var=False) # Welch's t-test
    print(f"  Overall Time Spent T-test: t={stat_overall:.3f}, p={p_val_overall:.3f}")
    if p_val_overall < alpha:
        print(f"  There is a statistically significant difference in overall time spent per question (p < {alpha}).")
    else:
        print(f"  There is no statistically significant difference in overall time spent per question (p >= {alpha}).")
else:
    print(f"  Not enough data to perform an overall t-test for time spent per question.")

print("\n--- T-tests for Time Spent per *Individual* Question between Group A and Group B ---")

t_test_results = []

# Get all unique question_uuids
unique_questions = individual_questions_df['question_uuid'].unique()

for q_uuid in unique_questions:
    time_group_a = individual_questions_df[
        (individual_questions_df['question_uuid'] == q_uuid) &
        (individual_questions_df['assigned_group'] == 'A')
    ]['timeSpent'].dropna()

    time_group_b = individual_questions_df[
        (individual_questions_df['question_uuid'] == q_uuid) &
        (individual_questions_df['assigned_group'] == 'B')
    ]['timeSpent'].dropna()

    # Only perform t-test if both groups have at least 2 data points
    if len(time_group_a) > 1 and len(time_group_b) > 1:
        stat, p_val = ttest_ind(time_group_a, time_group_b, equal_var=False) # Welch's t-test
        t_test_results.append({
            'question_uuid': q_uuid,
            't_statistic': stat,
            'p_value': p_val,
            'significant': 'Yes' if p_val < alpha else 'No'
        })
    else:
        t_test_results.append({
            'question_uuid': q_uuid,
            't_statistic': np.nan,
            'p_value': np.nan,
            'significant': 'Not enough data'
        })

t_test_df = pd.DataFrame(t_test_results)

significant_diffs = t_test_df[t_test_df['significant'] == 'Yes']
if not significant_diffs.empty:
    print(f"Number of questions with statistically significant differences in time spent: {len(significant_diffs)}")
else:
    print(f"No statistically significant differences found in time spent for any *individual* question (p >= {alpha}).")
```

```
--- Average Time Spent per Question by Group ---
```

assigned_group	mean	std
A	83.522137	143.825112
B	82.795862	107.059555

```
Average time spent per question by group calculated.

--- Overall T-test for Time Spent per Question between Group A and Group B ---
  Overall Time Spent T-test: t=0.106, p=0.916
  There is no statistically significant difference in overall time spent per question (p >= 0.05).

--- T-tests for Time Spent per *Individual* Question between Group A and Group B ---
No statistically significant differences found in time spent for any *individual* question (p >= 0.05).
```

This code block processes the ratings data embedded within the `questions` array of `completed_exercises_df`. It extracts 'clarity' and 'helpfulness' ratings for each question, associating them with the `userId`, `assigned_group`, and `actual_condition`. The resulting `ratings_df` is used for analyzing student perceptions of questions.

```
rating_rows = []

for _, row in completed_exercises_df.iterrows():
    qs = row["questions"]
```

```
    if not isinstance(qs, list):
        continue

    for q in qs:
        ratings = q.get("ratings", {})

        rating_rows.append({
            "userId": row["userId"],
            "assigned_group": row["assigned_group"],
            "actual_condition": row["actual_condition"],
            "clarity": ratings.get("clarity", np.nan),
            "helpfulness": ratings.get("helpfulness", np.nan)
        })

ratings_df = pd.DataFrame(rating_rows)
```

This section performs independent t-tests on the 'Clarity' and 'Helpfulness' ratings to determine if there are statistically significant differences in student perceptions between Group A (Self) and Group B (Peer). It also calculates Cohen's d as a measure of effect size for each comparison. A formatted HTML table with average ratings and tiny histograms is generated for a quick visual summary.

```
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
from IPython.display import HTML
from scipy.stats import ttest_ind

# Function to calculate Cohen's d
def cohens_d(x, y):
    nx = len(x)
    ny = len(y)
    dof = nx + ny - 2
    # Ensure there are enough degrees of freedom to avoid division by zero or negative dof
    if dof <= 0:
        return np.nan

    # Calculate pooled standard deviation
    # ddof=1 for sample standard deviation
    s1 = np.std(x, ddof=1)
    s2 = np.std(y, ddof=1)

    # Avoid division by zero if standard deviations are zero
    if s1 == 0 and s2 == 0:
        return 0.0 # If both std dev are 0, and means are different, this needs care.
                   # For now, if both are 0, Cohen's d is 0 (no effect size).

    pooled_std = np.sqrt(((nx - 1) * s1**2 + (ny - 1) * s2**2) / dof)

    if pooled_std == 0:
        return 0.0 # No variability, so no effect size difference if means are also same

    return (np.mean(x) - np.mean(y)) / pooled_std

# The ratings_df is already prepared in the notebook from previous cells.
# It contains 'userId', 'assigned_group', 'actual_condition', 'clarity', 'helpfulness'.

# Filter out users present in students_zero_exercises (from kernel state)
# `users_to_remove_from_feedback` is available from prior execution.
users_to_remove_from_feedback = students_zero_exercises['userId'].unique()
feedback_for_t_test_df = ratings_df[~ratings_df['userId'].isin(users_to_remove_from_feedback)].copy()

# Metrics for the current analysis
rating_cols_new = ["clarity", "helpfulness"]

# Melt the DataFrame (similar to feedback_plot_df creation)
plot_df_agg = feedback_for_t_test_df.melt(
    id_vars=["userId", "assigned_group"],
    value_vars=rating_cols_new,
    var_name="metric",
    value_name="rating"
)

plot_df_agg = plot_df_agg.dropna()
plot_df_agg["rating"] = plot_df_agg["rating"].round().astype(int)

# Function to convert camelCase to Title Case with spaces
def camel_to_title(text):
    s1 = re.sub(r'([a-z])([A-Z])', r'\\1 \\2', text)
    return s1.title()
```

```
plot_df_agg["metric"] = plot_df_agg["metric"].apply(camel_to_title)

# Custom metric titles for the table statements
custom_metric_titles_new = {
    "Clarity": "Clarity of Questions (1-5)",
    "Helpfulness": "Helpfulness of Questions (1-5)"
}

# Define group colors (consistent with previous cells, group_palette is in kernel state)

def create_tiny_histogram(ratings_series, color, x_min=1, x_max=5, bw_adjust=0.8):
    """
    Generates a tiny, smooth histogram (KDE plot) for a series of ratings.
    Returns a base64 encoded PNG image embedded in an HTML <img> tag.
    """
    if ratings_series.empty:
        return "No Data"

    fig, ax = plt.subplots(figsize=(1.5, 0.3), dpi=100) # Small figure size
    sns.kdeplot(ratings_series, ax=ax, color=color, fill=True, clip=(x_min, x_max), bw_adjust=bw_adjust)

    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_xlabel('')
    ax.set_ylabel('')
    ax.set_title('')
    for spine in ax.spines.values():
        spine.set_visible(False)
    ax.patch.set_alpha(0)

    ax.set_xlim(x_min, x_max)

    buf = io.BytesIO()
    plt.tight_layout(pad=0)
    plt.savefig(buf, format='png', bbox_inches='tight', pad_inches=0, transparent=True)
    plt.close(fig) # Close the figure to free memory
    image_base64 = base64.b64encode(buf.getvalue()).decode('utf-8')
    return f'<img src="{image_base64}" style="height:20px;">'


# Prepare data for the table
all_metrics_new = plot_df_agg['metric'].unique()

# Create a MultiIndex for columns
cols_new = pd.MultiIndex.from_product([['Group A', 'Group B'], ['Average Rating', 'Distribution']])
final_table_data_new = []

for metric_raw in all_metrics_new:
    statement_title = custom_metric_titles_new.get(metric_raw, metric_raw)
    row_data = {'Statement': statement_title}

    for group_name in ['A', 'B']:
        # Average Rating
        avg_rating = plot_df_agg[
            (plot_df_agg['metric'] == metric_raw) &
            (plot_df_agg['assigned_group'] == group_name)
        ]['rating'].mean().round(2)
        row_data[(f'Group {group_name}', 'Average Rating')] = avg_rating

        # Distribution (tiny histogram)
        group_ratings_series = plot_df_agg[
            (plot_df_agg['metric'] == metric_raw) &
            (plot_df_agg['assigned_group'] == group_name)
        ]['rating']

        hist_color = group_palette[group_name]
        histogram_html = create_tiny_histogram(group_ratings_series, hist_color)
        row_data[(f'Group {group_name}', 'Distribution')] = histogram_html
    final_table_data_new.append(row_data)

# Create the DataFrame
result_df_new = pd.DataFrame(final_table_data_new)
result_df_new = result_df_new.set_index('Statement')
result_df_new.columns = cols_new # Assign the MultiIndex


# --- Perform t-tests between Group A and Group B for each metric ---
print("\n--- T-tests for Clarity and Helpfulness Ratings between Group A and Group B ---")
alpha = 0.05

for metric_name in all_metrics_new:
    ratings_group_a = plot_df_agg[
        (plot_df_agg['metric'] == metric_name) &
        (plot_df_agg['assigned_group'] == 'A')
    ]['rating']
```

```
    ratings_group_b = plot_df_agg[
        (plot_df_agg['metric'] == metric_name) &
        (plot_df_agg['assigned_group'] == 'B')
    ]['rating']

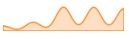
    if len(ratings_group_a) > 1 and len(ratings_group_b) > 1:
        stat, p_val = ttest_ind(ratings_group_a, ratings_group_b, equal_var=False)
        d = cohens_d(ratings_group_a, ratings_group_b) # Calculate Cohen's d
        print(f"\nMetric: {metric_name}")
        print(f"  T-test: t={stat:.3f}, p={p_val:.9f}, Cohen's d={d:.3f}")
        if p_val < alpha:
            print(f"  The difference is statistically significant (p < {alpha}).")
        else:
            print(f"  The difference is NOT statistically significant (p \\u2265 {alpha}).")
    else:
        print(f"\nMetric: {metric_name}")
        print(f"  Not enough data to perform a t-test for this metric.")

# Render as HTML
HTML(result_df_new.to_html(escape=False))
```

```
--- T-tests for Clarity and Helpfulness Ratings between Group A and Group B ---

Metric: Clarity
  T-test: t=-0.839, p=0.401661177, Cohen's d=-0.045
  The difference is NOT statistically significant (p \u2265 0.05).

Metric: Helpfulness
  T-test: t=-3.690, p=0.000233747, Cohen's d=-0.200
  The difference is statistically significant (p < 0.05).
```

	Group A		Group B	
Statement	Average Rating	Distribution	Average Rating	Distribution
Clarity of Questions (1-5)	3.54		3.59	
Helpfulness of Questions (1-5)	3.41		3.63	

This cell conducts a unique analysis by comparing the feedback ratings (Clarity and Helpfulness) for the *first* and *second* submissions made by students who submitted feedback multiple times. It identifies repeat submitters, sorts their submissions by date, extracts the first two, and then performs independent t-tests to see if there's a significant change in ratings over time for these metrics. This helps to understand if students' perceptions change after initial engagement.

```
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
from IPython.display import HTML
from scipy.stats import ttest_ind

# Function to calculate Cohen's d
def cohens_d(x, y):
    nx = len(x)
    ny = len(y)
    dof = nx + ny - 2
    # Ensure there are enough degrees of freedom to avoid division by zero or negative dof
    if dof <= 0:
        return np.nan

    # Calculate pooled standard deviation
    # ddof=1 for sample standard deviation
    s1 = np.std(x, ddof=1)
    s2 = np.std(y, ddof=1)

    # Avoid division by zero if standard deviations are zero
    if s1 == 0 and s2 == 0:
        # If both std dev are 0, Cohen's d is 0 if means are equal, else it's undefined (infinite effect size).
        # For practical purposes, if both are 0, Cohen's d is 0 (no effect size).
        return 0.0

    pooled_std = np.sqrt(((nx - 1) * s1**2 + (ny - 1) * s2**2) / dof)

    if pooled_std == 0:
        # If pooled std is 0, and means are different, this implies infinite effect size.
        # If means are also the same, then no effect. Returning 0.0 for cases where there is no variability
        # and no mean difference. Otherwise, it would be division by zero or infinite.
        if np.mean(x) != np.mean(y):
```

```
            return np.inf # Infinite effect size if means differ but no variance
        else:
            return 0.0 # No effect size if no variance and no mean difference

    return (np.mean(x) - np.mean(y)) / pooled_std

# Load feedback data
raw_feedback_data_df = pd.read_csv("all-study-feedback.csv")
raw_feedback_data_df = raw_feedback_data_df.rename(columns={user_id_column_name: "userId"})

# Merge with the users DataFrame to get assigned_group
# Assuming 'study_participants_df' DataFrame is already available from previous cells
processed_feedback_df = raw_feedback_data_df.merge(study_participants_df, on="userId", how="inner")

# Filter out users present in students_zero_exercises
users_to_remove_from_feedback = students_zero_exercises['userId'].unique()
processed_feedback_df = processed_feedback_df[~processed_feedback_df['userId'].isin(users_to_remove_from_feedback)]

# Columns for Likert scale ratings
rating_cols = [
    "easeOfUnderstanding",
    "reasonableQuestions",
    "helpsUnderstandCode",
    "helpsUnderstandJava"
]

# Average the ratings for each userId and assigned_group
aggregated_feedback_df = processed_feedback_df.groupby(["userId", "assigned_group"])[rating_cols].mean().reset_index()

# Melt the aggregated DataFrame
feedback_for_plotting_df = aggregated_feedback_df.melt(
    id_vars=["userId", "assigned_group"],
    value_vars=rating_cols,
    var_name="metric",
    value_name="rating"
)

feedback_for_plotting_df = feedback_for_plotting_df.dropna()
# Round ratings to the nearest integer as they are Likert scale
feedback_for_plotting_df["rating"] = feedback_for_plotting_df["rating"].round().astype(int)

# Function to convert camelCase to Title Case with spaces
def camel_to_title(text):
    s1 = re.sub(r'([a-z])([A-Z])', r'\1 \2', text) # Fixed the regex here
    return s1.title()

# Apply the improved capitalization for metric names
feedback_for_plotting_df["metric"] = feedback_for_plotting_df["metric"].apply(camel_to_title)

# Custom metric titles for the table statements
custom_metric_titles = {
    "Ease Of Understanding": "Q1: The exercises are easy to understand.",
    "Reasonable Questions": "Q4: The exercises are asking reasonable questions.",
    "Helps Understand Code": "Q2: The exercises are helping me better understand the programming assignment code they are based on.",
    "Helps Understand Java": "Q3: The exercises are overall helping me better understand the Java programming language."
}

# Define group colors (consistent with previous cells if group_palette exists)
group_palette = {
    "A": "#1f77b4",  # blue
    "B": "#ff7f0e"   # orange
}

def create_tiny_histogram(ratings_series, color, x_min=1, x_max=5, bw_adjust=0.8):
    """
    Generates a tiny, smooth histogram (KDE plot) for a series of ratings.
    Returns a base64 encoded PNG image embedded in an HTML <img> tag.
    """
    if ratings_series.empty:
        return "No Data"

    fig, ax = plt.subplots(figsize=(1.5, 0.3), dpi=100) # Small figure size
    # Use bw_adjust to control smoothness
    sns.kdeplot(ratings_series, ax=ax, color=color, fill=True, clip=(x_min, x_max), bw_adjust=bw_adjust)

    # Remove all plot decorations
    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_xlabel('')
    ax.set_ylabel('')
    ax.set_title('')
    for spine in ax.spines.values():
        spine.set_visible(False)
    ax.patch.set_alpha(0) # Make background transparent

    # Set x-limits to ensure 1-5 scale for all plots
```

```
    ax.set_xlim(x_min, x_max)

    buf = io.BytesIO()
    plt.tight_layout(pad=0)
    plt.savefig(buf, format='png', bbox_inches='tight', pad_inches=0, transparent=True)
    plt.close(fig) # Close the figure to free memory
    image_base64 = base64.b64encode(buf.getvalue()).decode('utf-8')
    return f'<img src="{image_base64}" style="height:20px;">'


# Prepare data for the table
all_metrics = feedback_for_plotting_df['metric'].unique()

# Create a MultiIndex for columns
cols = pd.MultiIndex.from_product([['Group A', 'Group B'], ['Average Rating', 'Distribution']])
final_table_data = []

for metric_raw in all_metrics:
    statement_title = custom_metric_titles.get(metric_raw, metric_raw)
    row_data = {'Statement': statement_title}

    for group_name in ['A', 'B']:
        # Average Rating
        avg_rating = feedback_for_plotting_df[
            (feedback_for_plotting_df['metric'] == metric_raw) &
            (feedback_for_plotting_df['assigned_group'] == group_name)
        ]['rating'].mean().round(2)
        row_data[(f'Group {group_name}', 'Average Rating')] = avg_rating

        # Distribution (tiny histogram)
        group_ratings_series = feedback_for_plotting_df[
            (feedback_for_plotting_df['metric'] == metric_raw) &
            (feedback_for_plotting_df['assigned_group'] == group_name)
        ]['rating']

        # Ensure consistent colors for groups A and B in the histograms
        hist_color = group_palette[group_name]
        histogram_html = create_tiny_histogram(group_ratings_series, hist_color)
        row_data[(f'Group {group_name}', 'Distribution')] = histogram_html
    final_table_data.append(row_data)

# Create the DataFrame
result_df = pd.DataFrame(final_table_data)
result_df = result_df.set_index('Statement')
result_df.columns = cols # Assign the MultiIndex


# --- Perform t-tests between Group A and Group B for each metric ---
print("\n--- T-tests for Feedback Ratings between Group A and Group B ---")
alpha = 0.05

for metric_name in all_metrics:
    ratings_group_a = feedback_for_plotting_df[
        (feedback_for_plotting_df['metric'] == metric_name) &
        (feedback_for_plotting_df['assigned_group'] == 'A')
    ]['rating']

    ratings_group_b = feedback_for_plotting_df[
        (feedback_for_plotting_df['metric'] == metric_name) &
        (feedback_for_plotting_df['assigned_group'] == 'B')
    ]['rating']

    # Perform independent t-test (Welch's t-test, assuming unequal variances)
    if len(ratings_group_a) > 1 and len(ratings_group_b) > 1: # Need at least 2 data points for t-test
        stat, p_val = ttest_ind(ratings_group_a, ratings_group_b, equal_var=False)
        d = cohens_d(ratings_group_a, ratings_group_b) # Calculate Cohen's d
        print(f"\nMetric: {metric_name}")
        print(f"  T-test: t={stat:.3f}, p={p_val:.3f}, Cohen's d={d:.3f}")
        if p_val < alpha:
            print(f"  The difference is statistically significant (p < {alpha}).")
        else:
            print(f"  The difference is NOT statistically significant (p \u2265 {alpha}).")
    else:
        print(f"\nMetric: {metric_name}")
        print(f"  Not enough data to perform a t-test for this metric.")

# Render as HTML, making sure to not escape the HTML image tags
HTML(result_df.to_html(escape=False))
```

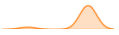
```
--- T-tests for Feedback Ratings between Group A and Group B ---

Metric: Ease Of Understanding
  T-test: t=-2.649, p=0.015, Cohen's d=-1.081
  The difference is statistically significant (p < 0.05).

Metric: Reasonable Questions
  T-test: t=-2.530, p=0.021, Cohen's d=-1.033
  The difference is statistically significant (p < 0.05).

Metric: Helps Understand Code
  T-test: t=-0.348, p=0.732, Cohen's d=-0.142
  The difference is NOT statistically significant (p ≥ 0.05).

Metric: Helps Understand Java
  T-test: t=-0.920, p=0.368, Cohen's d=-0.376
  The difference is NOT statistically significant (p ≥ 0.05).
```

	Group A		Group B	
Statement	Average Rating	Distribution	Average Rating	Distribution
Q1: The exercises are easy to understand.	3.08		3.83	
Q4: The exercises are asking reasonable questions.	3.58		4.25	
Q2: The exercises are helping me better understand the programming assignment code they are based on.	3.58		3.67	
Q3: The exercises are overall helping me better understand the Java programming language.	3.67		3.83	

```python
# --- New Analysis: T-tests for first vs. second submission for repeat submitters ---
print("\n--- T-tests for First vs. Second Feedback Submissions ---")

# Make a copy of raw_feedback_data_df for this analysis and ensure 'date' is datetime
multiple_submissions_feedback_df = raw_feedback_data_df.copy()
# Convert the 'date' column to datetime objects, assuming missing year is 2026.
# This appends '-2026' to date strings like '12-Mar' to provide a full date, then parses.
multiple_submissions_feedback_df['date'] = (
    multiple_submissions_feedback_df['date'].astype(str).apply(lambda x: f"{x}-2026" if not re.search(r'\\d{{4}}$', x) else x)
).apply(lambda x: pd.to_datetime(x, format='%d-%b-%Y', errors='coerce'))

# Sort by userId and date to ensure correct ordering of submissions
multiple_submissions_feedback_df = multiple_submissions_feedback_df.sort_values(by=['userId', 'date'])

# Count submissions per user
submission_counts = multiple_submissions_feedback_df.groupby('userId').size()

# Identify users who submitted more than once
repeat_submitters = submission_counts[submission_counts > 1].index

# Filter the DataFrame to only include submissions from repeat submitters
repeat_submitter_feedback_df = multiple_submissions_feedback_df[multiple_submissions_feedback_df['userId'].isin(repeat_submitters)].copy()

# Initialize lists to store first and second submission ratings
# Using 'easeOfUnderstanding' as proxy for 'clarity' and 'helpsUnderstandCode' for 'helpfulness'
first_clarity_ratings = []
second_clarity_ratings = []
first_helpfulness_ratings = []
second_helpfulness_ratings = []

# Iterate through each repeat submitter to get their first and second submissions
for user_id in repeat_submitters:
    user_submissions = repeat_submitter_feedback_df[repeat_submitter_feedback_df['userId'] == user_id]

    # Only consider users with at least two submissions after sorting
    if len(user_submissions) >= 2:
        # First submission
        first_sub = user_submissions.iloc[0]
        first_clarity_ratings.append(first_sub['easeOfUnderstanding']) # Using easeOfUnderstanding as clarity proxy
        first_helpfulness_ratings.append(first_sub['helpsUnderstandCode']) # Using helpsUnderstandCode as helpfulness proxy

        # Second submission
        second_sub = user_submissions.iloc[1]
        second_clarity_ratings.append(second_sub['easeOfUnderstanding']) # Using easeOfUnderstanding as clarity proxy
        second_helpfulness_ratings.append(second_sub['helpsUnderstandCode']) # Using helpsUnderstandCode as helpfulness proxy

# Convert to numpy arrays or pandas Series for t-test
first_clarity = pd.Series(first_clarity_ratings).dropna()
second_clarity = pd.Series(second_clarity_ratings).dropna()

first_helpfulness = pd.Series(first_helpfulness_ratings).dropna()
```

```
second_helpfulness = pd.Series(second_helpfulness_ratings).dropna()

# Perform t-tests
alpha = 0.05 # Significance level

print("\nMetric: Clarity (Ease of Understanding - First vs. Second Submission)")
if len(first_clarity) > 1 and len(second_clarity) > 1:
    stat_clarity, p_val_clarity = ttest_ind(first_clarity, second_clarity, equal_var=False)
    print(f"  T-test: t={stat_clarity:.3f}, p={p_val_clarity:.9f}")
    if p_val_clarity < alpha:
        print(f"  The difference is statistically significant (p < {alpha}).")
    else:
        print(f"  The difference is NOT statistically significant (p \u2265 {alpha}).")
else:
    print("  Not enough data to perform a t-test for Clarity (Ease of Understanding - First vs. Second Submission).")

print("\nMetric: Helpfulness (Helps Understand Code - First vs. Second Submission)")
if len(first_helpfulness) > 1 and len(second_helpfulness) > 1:
    stat_helpfulness, p_val_helpfulness = ttest_ind(first_helpfulness, second_helpfulness, equal_var=False)
    print(f"  T-test: t={stat_helpfulness:.3f}, p={p_val_helpfulness:.9f}")
    if p_val_helpfulness < alpha:
        print(f"  The difference is statistically significant (p < {alpha}).")
    else:
        print(f"  The difference is NOT statistically significant (p \u2265 {alpha}).")
else:
    print("  Not enough data to perform a t-test for Helpfulness (Helps Understand Code - First vs. Second Submission).")
```

```
--- T-tests for First vs. Second Feedback Submissions ---

Metric: Clarity (Ease of Understanding - First vs. Second Submission)
  T-test: t=-0.593, p=0.557589608
  The difference is NOT statistically significant (p ≥ 0.05).

Metric: Helpfulness (Helps Understand Code - First vs. Second Submission)
  T-test: t=-0.855, p=0.399648867
  The difference is NOT statistically significant (p ≥ 0.05).
```

Exam Score Comparisons

This block loads exam scores from `spring2026.csv` and performs a complex merging and filtering process:

- Renames `idCol` to `userId`.
- Merges with `users` (study participants) and `all_users_df` (all students) to get `assigned_group` and `studyParticipation` status.
- Merges with `summary_table` to get `Total Completed Exercises`.
- **Crucially, it references `user_ids_to_remove` (from prior analysis) to define students who should be removed from the final analysis.**
- **Defines `final_group`**: categorizes students into 'A', 'B', or 'C' (baseline) based on their `assigned_group` and `Total Completed Exercises`, also marking some for removal.
- Drops students identified for removal and those in Group C with 0 or NaN exam scores.
- Calculates `exam1_2_avg`.

This rigorous filtering ensures that only valid student records are used for exam score comparisons.

```
import pandas as pd
import numpy as np
from scipy.stats import ttest_ind, mannwhitneyu # Changed to ttest_ind and added mannwhitneyu for independent samples

csv_path = 'spring2026.csv'
exam_scores_df = pd.read_csv(csv_path)

# Rename 'idCol' to 'userId' for merging with the 'study_participants_df' DataFrame
exam_scores_df = exam_scores_df.rename(columns={user_id_column_name: 'userId'})

# Merge with the existing 'study_participants_df' DataFrame (filtered for study participants A and B)
# A left merge ensures all students from the CSV are kept
exam_scores_with_groups_df = exam_scores_df.merge(study_participants_df[['userId', 'assigned_group']], on='userId', how='left')

# MERGE studyParticipation status from all_users_df
# all_users_df is already filtered for role == "student" and contains studyParticipation
exam_scores_with_groups_df = exam_scores_with_groups_df.merge(
    all_users_df[['userId', 'studyParticipation']],
    on='userId',
    how='left',
    suffixes=('_scores', '_allusers')
)
# Fill NaN for studyParticipation: if not found in all_users_df, they are not in qlcs.users.json -> studyParticipation is effectively False
exam_scores_with_groups_df['studyParticipation'] = exam_scores_with_groups_df['studyParticipation'].fillna(False)
```

```
# Merge Total Completed Exercises into exam_scores_with_groups_df to identify students with no completion
# `participation_summary_table_df` is generated in cell `c7bInE6RygGx` and contains 'Total Completed Exercises'
exam_scores_with_groups_df = exam_scores_with_groups_df.merge(
    participation_summary_table_df[['userId', 'Total Completed Exercises']],
    on='userId',
    how='left'
)
# Fill NaN for 'Total Completed Exercises' with 0 (for students not in A/B, or those who truly completed 0)
exam_scores_with_groups_df['Total Completed Exercises'] = exam_scores_with_groups_df['Total Completed Exercises'].fillna(0).astype(int)

# Students to be completely removed from analysis are those who had studyParticipation = False
# but still completed exercises (this is `user_ids_to_remove` from `mtG7ST8CktpK`).
final_analysis_removed_ids = set(user_ids_to_remove)

# Define conditions for final_group assignment based on the updated requirements
conditions = [
    # Condition 1: Students to be removed from analysis
    exam_scores_with_groups_df['userId'].isin(final_analysis_removed_ids),
    # Condition 2: Students who completed zero exercises (assign to 'C')
    (exam_scores_with_groups_df['Total Completed Exercises'] == 0),
    # Condition 3: Students originally A/B and completed >0 exercises (keep A/B assignment)
    exam_scores_with_groups_df['assigned_group'].isin(['A', 'B'])
]

# Define choices corresponding to the conditions
choices = [
    pd.NA,  # Mark for removal
    'C',    # Assign to 'C'
    exam_scores_with_groups_df['assigned_group'] # Keep original A/B assignment
]

# Apply the conditions using np.select
exam_scores_with_groups_df['final_group'] = np.select(
    conditions,
    choices,
    default='C' # Default for students who are not removed, did not complete 0 exercises, and were not originally A/B (i.e., original C with >0
)

# Drop students who are marked for removal (final_group is pd.NA)
exam_scores_with_groups_df.dropna(subset=['final_group'], inplace=True)

# Drop the temporary columns used for filtering ('Total Completed Exercises' and 'studyParticipation')
exam_scores_with_groups_df.drop(columns=['Total Completed Exercises', 'studyParticipation'], inplace=True)

# Ensure score columns are numeric (handle potential non-numeric data gracefully)
score_cols = ['Exam 1 Points Grade <Numeric MaxPoints:100 Weight:10>', 'Exam 2 Points Grade <Numeric MaxPoints:100 Weight:10>', 'Exam 3 Points
for col in score_cols:
    exam_scores_with_groups_df[col] = pd.to_numeric(exam_scores_with_groups_df[col], errors='coerce')

# Calculate the average of exam1 and exam2
exam_scores_with_groups_df['exam1_2_avg'] = exam_scores_with_groups_df[['Exam 1 Points Grade <Numeric MaxPoints:100 Weight:10>', 'Exam 2 Points

# Count and display students in Group C (baseline group)
group_c_students = exam_scores_with_groups_df[exam_scores_with_groups_df['final_group'] == 'C']
num_group_c_students = group_c_students['userId'].nunique()
print(f"\nNumber of students in the baseline group (Group C): {num_group_c_students}")

# Define exam columns to check for 0 or NaN scores
exam_columns_to_check = [
    'Exam 1 Points Grade <Numeric MaxPoints:100 Weight:10>',
    'Exam 2 Points Grade <Numeric MaxPoints:100 Weight:10>',
    'Exam 3 Points Grade <Numeric MaxPoints:100 Weight:10>',
    'Final Exam Points Grade <Numeric MaxPoints:100 Weight:15>'
]

# Identify students in Group C with 0 or NaN in any of the exam columns
students_to_drop_from_c = group_c_students[
    (group_c_students[exam_columns_to_check].isnull()).any(axis=1) | # Check for any NaN
    ((group_c_students[exam_columns_to_check] == 0)).any(axis=1)    # Check for any 0
]['userId'].unique()

# Drop these students from the main exam_scores_with_groups_df DataFrame
exam_scores_with_groups_df = exam_scores_with_groups_df[~exam_scores_with_groups_df['userId'].isin(students_to_drop_from_c)].copy()

# Recount and display students in Group C after dropping
group_c_students_after_drop = exam_scores_with_groups_df[exam_scores_with_groups_df['final_group'] == 'C']
num_group_c_students_after_drop = group_c_students_after_drop['userId'].nunique()
print(f"Number of students in the baseline group (Group C) after dropping: {num_group_c_students_after_drop}")
```

```
Number of students in the baseline group (Group C): 57
Number of students in the baseline group (Group C) after dropping: 56
```

This cell continues the filtering of exam data, specifically for students in Group A or B who might have 0 or NaN scores in any of the exam columns. It explicitly drops these students from the `exam_scores_with_groups_df` DataFrame. It then calculates the mean and

standard deviation of various exam scores for each `final_group` (A, B, C), presenting them in a formatted table as 'Mean ± Std Dev'. Zero scores are treated as `np.nan` for accurate average calculation.

```
# Define the columns for which to calculate averages
analysis_cols = ['exam1_2_avg', 'Exam 3 Points Grade <Numeric MaxPoints:100 Weight:10>', 'Final Exam Points Grade <Numeric MaxPoints:100 Weight

# Define exam columns to check for 0 or NaN scores (already defined in previous cell, but re-defining for clarity here)
exam_columns_to_check = [
    'Exam 1 Points Grade <Numeric MaxPoints:100 Weight:10>',
    'Exam 2 Points Grade <Numeric MaxPoints:100 Weight:10>',
    'Exam 3 Points Grade <Numeric MaxPoints:100 Weight:10>',
]

# Identify students in Group A or B with 0 or NaN in any of the exam columns
students_in_a_b = exam_scores_with_groups_df[exam_scores_with_groups_df['final_group'].isin(['A', 'B'])].copy()

students_to_drop_from_a_b = students_in_a_b[
    (students_in_a_b[exam_columns_to_check].isnull()).any(axis=1) | # Check for any NaN
    ((students_in_a_b[exam_columns_to_check] == 0)).any(axis=1)    # Check for any 0
][['userId', 'final_group']]

# Print the IDs and assigned groups of students to be dropped from A/B
if not students_to_drop_from_a_b.empty:
    print(f"Number of students from Group A or B to be dropped due to 0 or NaN in exams: {len(students_to_drop_from_a_b)}")

# Drop these students from the main exam_scores_with_groups_df DataFrame
exam_scores_for_analysis_df = exam_scores_with_groups_df[~exam_scores_with_groups_df['userId'].isin(students_to_drop_from_a_b['userId'])].copy(

# Create a copy of exam_scores_for_analysis_df to avoid modifying the original DataFrame
# and replace 0 scores with NaN in the analysis columns for proper aggregation (excluding zeros).
# The name `exam_scores_for_analysis_df` already implies it's the filtered one, so no intermediate copy needed.
for col in analysis_cols:
    exam_scores_for_analysis_df[col] = exam_scores_for_analysis_df[col].replace(0, np.nan)

# Calculate both means and standard deviations for each final group
# Use the filtered DataFrame
grouped_stats = exam_scores_for_analysis_df.groupby('final_group')[analysis_cols].agg(['mean', 'std'])

# Prepare a new DataFrame to store the formatted output
formatted_averages = pd.DataFrame(index=grouped_stats.index)

# Define mapping for original column names to desired display names
display_name_map = {
    'exam1_2_avg': 'Exam 1 & 2',
    'Exam 3 Points Grade <Numeric MaxPoints:100 Weight:10>': 'Exam 3',
    'Final Exam Points Grade <Numeric MaxPoints:100 Weight:15>': 'Final Exam',
}

# Loop through each analysis column to create the 'Mean +/- Std Dev' format
for col_original, col_display in display_name_map.items():
    mean_val = grouped_stats[col_original]['mean']
    std_val = grouped_stats[col_original]['std']
    # Format as "Mean +/- Std Dev" with 2 decimal places
    formatted_averages[f'{col_display}'] = mean_val.round(2).astype(str) + ' \u00b1 ' + std_val.round(2).astype(str)

# Reorder rows to A, B, C for consistent display
final_table = formatted_averages.reindex(['A', 'B', 'C'])

# Rename the index name to 'Group'
final_table.index.name = 'Group'

display(final_table)
```

Group	Exam 1 & 2	Exam 3	Final Exam
A	81.16 ± 12.4	78.97 ± 13.78	82.73 ± 12.76
B	83.72 ± 6.25	83.61 ± 9.34	83.24 ± 8.52
C	80.64 ± 11.31	78.18 ± 16.24	76.35 ± 18.69

T-Tests between Groups

This comprehensive cell conducts independent t-tests (Welch's t-test, assuming unequal variances) for all pairwise comparisons between groups (A vs. B, A vs. C, B vs. C) across all defined exam and final grade metrics. It explicitly removes students with 0 exam scores from the statistical analysis to avoid skewing results. For each test, it prints the t-statistic, p-value, Cohen's d, and a statement on statistical significance.

```
# --- Statistical Significance Testing (Independent t-tests between groups) ---

alpha = 0.05 # Significance level

print("\n--- Statistical Significance Testing (Independent t-tests between groups) ---")

groups = ['A', 'B', 'C']

# The exam_scores_for_analysis_df is already filtered and cleaned in the previous cell.
# Students with 0 or NaN scores in relevant exam columns have been handled, and 0s converted to NaN.

# Iterate through each analysis column (exam/grade instance)
for col_original, col_display in display_name_map.items():
    print(f"\nAnalysis for '{col_display}':")

    # Iterate through pairwise combinations of groups
    for i in range(len(groups)):
        for j in range(i + 1, len(groups)):
            group1_name = groups[i]
            group2_name = groups[j]

            # Use exam_scores_for_analysis_df for the analysis
            data_group1 = exam_scores_for_analysis_df[exam_scores_for_analysis_df['final_group'] == group1_name][col_original].dropna()
            data_group2 = exam_scores_for_analysis_df[exam_scores_for_analysis_df['final_group'] == group2_name][col_original].dropna()

            # Perform independent t-test if both groups have enough data
            if len(data_group1) > 1 and len(data_group2) > 1:
                stat, p_val = ttest_ind(data_group1, data_group2, equal_var=False) # Welch's t-test for unequal variances
                d = cohens_d(data_group1, data_group2) # Calculate Cohen's d
                print(f"  Comparison: Group {group1_name} vs Group {group2_name}")
                print(f"    Independent t-test: t={stat:.3f}, p={p_val:.3f}, Cohen's d={d:.3f}")
                if p_val < alpha:
                    print(f"    The difference is statistically significant (p < {alpha}).")
                else:
                    print(f"    The difference is NOT statistically significant (p \u2265 {alpha}).")
            else:
                print(f"  Not enough valid data for Group {group1_name} vs Group {group2_name} for '{col_display}'.")
```

```
--- Statistical Significance Testing (Independent t-tests between groups) ---

Analysis for 'Exam 1 & 2':
  Comparison: Group A vs Group B
    Independent t-test: t=-0.751, p=0.461, Cohen's d=-0.269
    The difference is NOT statistically significant (p ≥ 0.05).
  Comparison: Group A vs Group C
    Independent t-test: t=0.150, p=0.882, Cohen's d=0.045
    The difference is NOT statistically significant (p ≥ 0.05).
  Comparison: Group B vs Group C
    Independent t-test: t=1.481, p=0.144, Cohen's d=0.300
    The difference is NOT statistically significant (p ≥ 0.05).

Analysis for 'Exam 3':
  Comparison: Group A vs Group B
    Independent t-test: t=-1.143, p=0.264, Cohen's d=-0.401
    The difference is NOT statistically significant (p ≥ 0.05).
  Comparison: Group A vs Group C
    Independent t-test: t=0.194, p=0.848, Cohen's d=0.050
    The difference is NOT statistically significant (p ≥ 0.05).
  Comparison: Group B vs Group C
    Independent t-test: t=1.779, p=0.081, Cohen's d=0.366
    The difference is NOT statistically significant (p ≥ 0.05).

Analysis for 'Final Exam':
  Comparison: Group A vs Group B
    Independent t-test: t=-0.131, p=0.897, Cohen's d=-0.048
    The difference is NOT statistically significant (p ≥ 0.05).
  Comparison: Group A vs Group C
    Independent t-test: t=1.545, p=0.132, Cohen's d=0.362
    The difference is NOT statistically significant (p ≥ 0.05).
  Comparison: Group B vs Group C
    Independent t-test: t=2.172, p=0.033, Cohen's d=0.411
    The difference is statistically significant (p < 0.05).
```

```
import glob
import re
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure group_palette is defined if not already in global scope
group_palette = {
    "A": "#1f77b4",  # blue
    "B": "#ff7f0e",   # orange
    "C": "#2ca02c"    # green for Group C
}

# 1. Prepare user_data_for_mapping
```

```
# 1. Prepare user data for mapping
# The 'study_participants_df' DataFrame already contains 'userId' (which is idCol) and 'assign
# We need to add the 'email' from raw_users_data for mapping with CSV 'Username' column.

id_col_to_email_map = {user_data[user_id_column_name]: user_data['email'] for user_data in raw

# Add email to the 'study_participants_df' DataFrame
# The 'study_participants_df' DataFrame has 'userId' which is the 'idCol'
study_participants_df['email'] = study_participants_df['userId'].map(id_col_to_email_map)

# Create a lookup DataFrame containing email, userId, and assigned_group for study participant
user_group_mapping = study_participants_df[['email', 'userId', 'assigned_group']].copy()


# 2. Load and process all CSVs in 'pas' folder
all_csv_data = []
pas_files = glob.glob("/content/pas/*.csv")

for file_path in pas_files:
    df_temp = pd.read_csv(file_path)

    # Extract assignment identifier (e.g., "PA06-A") from filename
    filename = file_path.split('/')[-1]
    assignment_id_match = re.match(r'(PA\d{2}-[A-Za-z]+)', filename)
    if assignment_id_match:
        assignment_id = assignment_id_match.group(1)
        assignment_group = assignment_id.split('-')[0] # e.g., PA06 from PA06-A
    else:
        # Skip files that don't match the expected pattern
        print(f"Warning: Skipping file {filename} due to unexpected naming pattern.")
        continue

    # Rename 'Username' to 'email' for merging
    df_temp = df_temp.rename(columns={'Username': 'email'})

    # Calculate score, handle division by zero or missing columns
    if 'Correctness Total' in df_temp.columns and 'Correctness Total Possible' in df_temp.colu
        df_temp['Correctness Total Possible'] = pd.to_numeric(df_temp['Correctness Total Possi
        df_temp['Correctness Total'] = pd.to_numeric(df_temp['Correctness Total'], errors='coe

        df_temp['score'] = np.where(
            df_temp['Correctness Total Possible'] > 0,
            df_temp['Correctness Total'] / df_temp['Correctness Total Possible'],
            0 # Assign 0 if possible score is 0 to avoid NaN/Inf
        )
    else:
        print(f"Warning: Missing 'Correctness Total' or 'Correctness Total Possible' in {filen
        df_temp['score'] = np.nan # Assign NaN score if columns are missing

    # Select relevant columns and append
    all_csv_data.append(df_temp[['email', 'score']].assign(
        assignment_id=assignment_id,
        assignment_group=assignment_group
    ))

# Concatenate all data into a single DataFrame
pa_scores_df = pd.concat(all_csv_data, ignore_index=True)

# Drop rows where score is NaN (e.g., from missing columns or invalid calculation or missing e
pa_scores_df.dropna(subset=['score', 'email'], inplace=True)

# 3. Merge with user groups
# Merge pa_scores_df with user_group_mapping to get assigned_group
# Using how='left' to keep all students from the CSVs
pa_scores_with_groups_df = pa_scores_df.merge(user_group_mapping[['email', 'assigned_group']],

# Assign 'C' to students not in Group A or B
pa_scores_with_groups_df['final_group'] = pa_scores_with_groups_df['assigned_group'].fillna('C

# 4. Aggregate scores for the summary table
pa_overall_score_summary_df = pa_scores_with_groups_df.groupby('final_group').agg(
    mean_pct=('score', lambda x: x.mean() * 100), # Mean as percentage
    std_pct=('score', lambda x: x.std() * 100)    # Standard deviation as percentage
).round(2)

print("Overall Percentage of Test Cases Passed Summary:")
display(pa_overall_score_summary_df)
```

```
Overall Percentage of Test Cases Passed Summary:

             mean_pct  std_pct
final_group
A               88.92    29.40
B               97.87     7.08
C               96.28    12.29
```

This cell performs the following steps:

1. **Prepare User Data**: Maps `userId` to `email` and creates a `user_group_mapping` DataFrame.
2. **Load and Process PAs**: Iterates through all CSV files in the `/content/pas/` folder, extracts `assignment_id` and `assignment_group` from filenames, calculates `score` (percentage of correctness), and concatenates them into `pa_scores_df`.
3. **Merge with User Groups**: Merges `pa_scores_df` with `user_group_mapping` to assign `final_group` (A, B, or C) to each entry.
4. **Aggregate Scores**: Calculates the mean and standard deviation of pass percentages for each `final_group`, presenting an `pa_overall_score_summary_df` table.
5. **Removal of previous visualizations**: The previous line plot generation code has been commented out as it is no longer requested, ensuring the focus remains on aggregated results.

Statistical Tests for Programmatic Assignment Pass Percentages

This section conducts independent t-tests to determine if there are statistically significant differences in the programmatic assignment pass percentages between the study groups (A, B, and C).

```
from scipy.stats import ttest_ind

alpha = 0.05

print("--- Independent T-tests for Pass Percentages between Groups ---")

# Extract scores for each group
group_a_pa_scores = pa_scores_with_groups_df[pa_scores_with_groups_df['final_group'] == 'A']['score'].dropna()
group_b_pa_scores = pa_scores_with_groups_df[pa_scores_with_groups_df['final_group'] == 'B']['score'].dropna()
group_c_pa_scores = pa_scores_with_groups_df[pa_scores_with_groups_df['final_group'] == 'C']['score'].dropna()

groups_data = {
    'A': group_a_pa_scores,
    'B': group_b_pa_scores,
    'C': group_c_pa_scores
}

pairwise_comparisons = [('A', 'B'), ('A', 'C'), ('B', 'C')]

for group1_name, group2_name in pairwise_comparisons:
    data_group1 = groups_data[group1_name]
    data_group2 = groups_data[group2_name]

    if len(data_group1) > 1 and len(data_group2) > 1:
        stat, p_val = ttest_ind(data_group1, data_group2, equal_var=False) # Welch's t-test for unequal variances
        print(f"\n  Comparison: Group {group1_name} vs Group {group2_name}")
        print(f"    Independent t-test: t={stat:.3f}, p={p_val:.3f}")
        if p_val < alpha:
            print(f"    The difference is statistically significant (p < {alpha}).")
        else:
            print(f"    The difference is NOT statistically significant (p \u2265 {alpha}).")
    else:
        print(f"\n  Not enough data to perform a t-test for Group {group1_name} vs Group {group2_name}.")
```

```
--- Independent T-tests for Pass Percentages between Groups ---

  Comparison: Group A vs Group B
    Independent t-test: t=-0.883, p=0.377
    The difference is NOT statistically significant (p ≥ 0.05).

  Comparison: Group A vs Group C
    Independent t-test: t=-1.329, p=0.184
    The difference is NOT statistically significant (p ≥ 0.05).

  Comparison: Group B vs Group C
    Independent t-test: t=-0.302, p=0.763
    The difference is NOT statistically significant (p ≥ 0.05).
```

This cell performs independent t-tests for pairwise comparisons of `score` (pass percentages) between Group A, Group B, and Group C. For each comparison, it prints the t-statistic, p-value, and a conclusion regarding statistical significance. This helps determine if the study interventions had a significant impact on programmatic assignment performance.